

1. Write a multithreaded program where multiple threads increment a shared counter. Use a mutex to ensure mutual exclusion so that only one thread can modify the counter at a time. Demonstrate how the absence of a mutex leads to race conditions and incorrect results, and how using a mutex ensures correct synchronization.

```

1 #include <stdio.h>
2 #include <pthread.h>
3
4 #define NUM_THREADS 10
5 #define INCREMENTS_PER_THREAD 100000
6
7 int counter = 0;
8 pthread_mutex_t lock;
9
10 void* unsafe_increment(void* arg) {
11     for(int i = 0; i < INCREMENTS_PER_THREAD; i++) {
12         counter++; // Race condition occurs here
13     }
14     return NULL;
15 }
16
17 void* safe_increment(void* arg) {
18     for(int i = 0; i < INCREMENTS_PER_THREAD; i++) {
19         pthread_mutex_lock(&lock);
20         counter++; // Protected critical section
21         pthread_mutex_unlock(&lock);
22     }
23     return NULL;
24 }
25
26 int main() {
27     pthread_t threads[NUM_THREADS];
28     pthread_mutex_init(&lock, NULL);
29
30     // Demonstration 1: Unsafe
31     printf("Running unsafe increment with %d threads...\n", NUM_THREADS);
32     for(int i = 0; i < NUM_THREADS; i++) {
33         pthread_create(&threads[i], NULL, unsafe_increment, NULL);
34     }
35     for(int i = 0; i < NUM_THREADS; i++) {
36         pthread_join(threads[i], NULL);
37     }
38     printf("Final Unsafe Counter: %d (Expected: %d)\n", counter, NUM_THREADS * INCREMENTS_PER_THREAD);
39 }

```

Bracket match found on line: 17

C Tab Width: 8 Ln 24, Col 2 INS

```

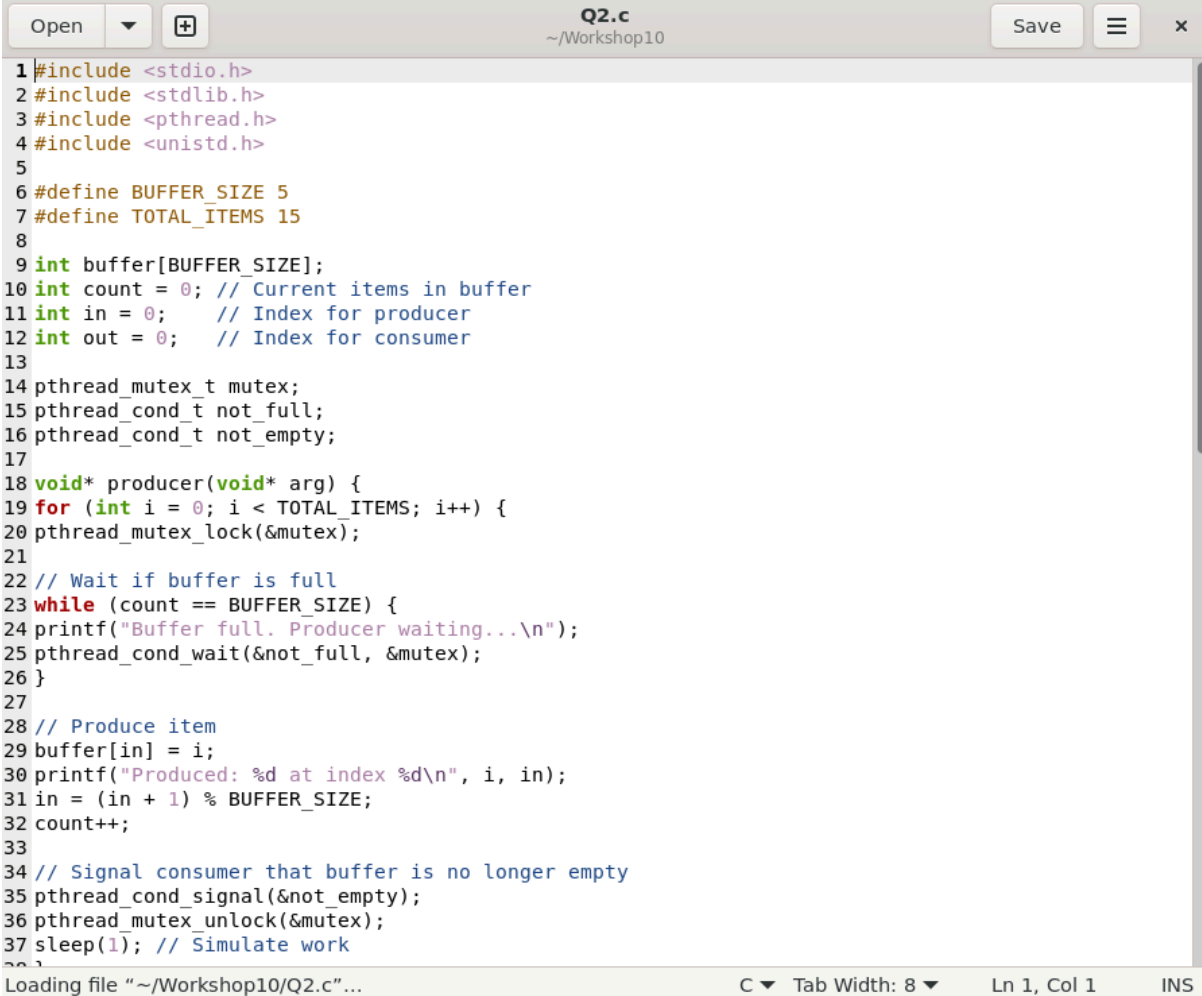
acer@Ashmit:~/Workshop10$ gcc Q1.c -o Q1
acer@Ashmit:~/Workshop10$ ./Q1
Running unsafe increment with 10 threads...
Final Unsafe Counter: 290270 (Expected: 1000000)

Running safe increment with mutex...
Final Safe Counter: 1000000 (Expected: 1000000)
acer@Ashmit:~/Workshop10$ gedit Q1.c

```

2. Write a multithreaded program to solve the Producer–Consumer problem using a bounded buffer of fixed size. Use a mutex to ensure mutual exclusion

when accessing the shared buffer. Use two condition variables: `not_full` to wait when the buffer is full and `not_empty` to wait when the buffer is empty. Ensure proper synchronization between producer and consumer threads without busy waiting.



```
Q2.c
~/Workshop10
Save

1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <pthread.h>
4 #include <unistd.h>
5
6 #define BUFFER_SIZE 5
7 #define TOTAL_ITEMS 15
8
9 int buffer[BUFFER_SIZE];
10 int count = 0; // Current items in buffer
11 int in = 0;    // Index for producer
12 int out = 0;   // Index for consumer
13
14 pthread_mutex_t mutex;
15 pthread_cond_t not_full;
16 pthread_cond_t not_empty;
17
18 void* producer(void* arg) {
19     for (int i = 0; i < TOTAL_ITEMS; i++) {
20         pthread_mutex_lock(&mutex);
21
22         // Wait if buffer is full
23         while (count == BUFFER_SIZE) {
24             printf("Buffer full. Producer waiting...\n");
25             pthread_cond_wait(&not_full, &mutex);
26         }
27
28         // Produce item
29         buffer[in] = i;
30         printf("Produced: %d at index %d\n", i, in);
31         in = (in + 1) % BUFFER_SIZE;
32         count++;
33
34         // Signal consumer that buffer is no longer empty
35         pthread_cond_signal(&not_empty);
36         pthread_mutex_unlock(&mutex);
37         sleep(1); // Simulate work
38     }
39 }
```

Loading file "~/Workshop10/Q2.c"...

C ▾ Tab Width: 8 ▾ Ln 1, Col 1 INS

```

acer@Ashmit:~/Workshop10$ gcc Q2.c -o Q2
acer@Ashmit:~/Workshop10$ ./Q2
Produced: 0 at index 0
Consumed: 0 from index 0
Produced: 1 at index 1
Consumed: 1 from index 1
Produced: 2 at index 2
Produced: 3 at index 3
Consumed: 2 from index 2
Produced: 4 at index 4
Produced: 5 at index 0
Consumed: 3 from index 3
Produced: 6 at index 1
Produced: 7 at index 2
Consumed: 4 from index 4
Produced: 8 at index 3
Produced: 9 at index 4
Consumed: 5 from index 0
Produced: 10 at index 0
Buffer full. Producer waiting...
Consumed: 6 from index 1
Produced: 11 at index 1
Buffer full. Producer waiting...
Consumed: 7 from index 2
Produced: 12 at index 2
Buffer full. Producer waiting...
Consumed: 8 from index 3

```

3. Write a multithreaded program to solve the Producer–Consumer problem using a bounded buffer of fixed size. Use semaphores to synchronize access to the buffer, including a counting semaphore to track empty slots and another to track filled slots. Ensure mutual exclusion while accessing the buffer using a binary semaphore. The producer must wait when the buffer is full, and the consumer must wait when the buffer is empty, without using busy waiting.

Open ▾ +

Q3.c
~/Workshop10

Save ≡ ×

```
1 #include <stdio.h>
2 #include <pthread.h>
3 #include <semaphore.h>
4 #include <unistd.h>
5 #include <stdlib.h>
6
7 #define BUFFER_SIZE 5
8 #define TOTAL_ITEMS 10
9
10 int buffer[BUFFER_SIZE];
11 int in = 0;
12 int out = 0;
13
14 sem_t empty;
15 sem_t full;
16 sem_t mutex;
17
18 void* producer(void* arg) {
19     for(int i = 0; i < TOTAL_ITEMS; i++) {
20         // Wait for an empty slot
21         sem_wait(&empty);
22         // Lock the buffer
23         sem_wait(&mutex);
24
25         // Critical Section
26         buffer[in] = i + 100;
27         printf("Producer: Inserted %d at %d\n", buffer[in], in);
28         in = (in + 1) % BUFFER_SIZE;
29
30         // Unlock and signal that an item is ready
31         sem_post(&mutex);
32         sem_post(&full);
33
34         sleep(1);
35     }
36     return NULL;
37 }
```

Saving file "~/Workshop10/Q3.c"...

C ▾ Tab Width: 8 ▾ Ln 81, Col 2 INS

```

acer@Ashmit:~/Workshop10$ gcc Q3.c -o Q3
acer@Ashmit:~/Workshop10$ ./Q3
Producer: Inserted 1 at index 0
Consumer: Removed 1 from index 0
Producer: Inserted 2 at index 1
Consumer: Removed 2 from index 1
Producer: Inserted 3 at index 2
Producer: Inserted 4 at index 3
Consumer: Removed 3 from index 2
Producer: Inserted 5 at index 4
Producer: Inserted 6 at index 0
Consumer: Removed 4 from index 3
Producer: Inserted 7 at index 1
Producer: Inserted 8 at index 2
Consumer: Removed 5 from index 4
Producer: Inserted 9 at index 3
Producer: Inserted 10 at index 4
Consumer: Removed 6 from index 0
Consumer: Removed 7 from index 1
Consumer: Removed 8 from index 2
Consumer: Removed 9 from index 3
Consumer: Removed 10 from index 4
Simulation complete.
acer@Ashmit:~/Workshop10$

```

4. Create a multithreaded program that applies a sepia filter to an image. This is the process of recalculating each pixel's RGB values using the sepia conversion formula.

Formula:

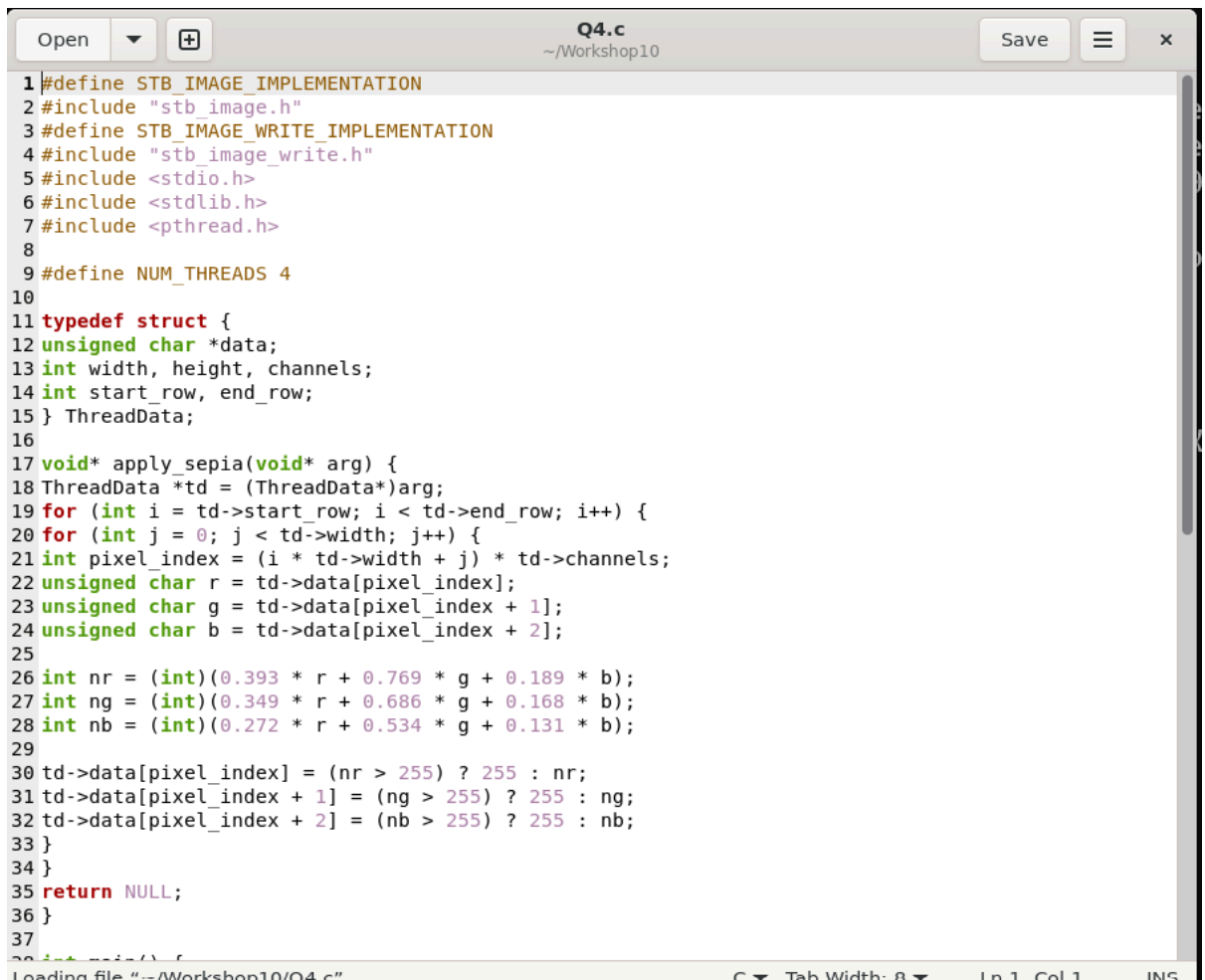
$$\text{NewR} = 0.393\text{R} + 0.769\text{G} + 0.189\text{B}$$

$$\text{NewG} = 0.349\text{R} + 0.686\text{G} + 0.168\text{B}$$

$$\text{NewB} = 0.272\text{R} + 0.534\text{G} + 0.131\text{B}$$

If any value exceeds 255, set it to 255.

NOTE: 0 is the minimum, and 255 is the maximum.



```
1 #define STB_IMAGE_IMPLEMENTATION
2 #include "stb_image.h"
3 #define STB_IMAGE_WRITE_IMPLEMENTATION
4 #include "stb_image_write.h"
5 #include <stdio.h>
6 #include <stdlib.h>
7 #include <pthread.h>
8
9 #define NUM_THREADS 4
10
11 typedef struct {
12     unsigned char *data;
13     int width, height, channels;
14     int start_row, end_row;
15 } ThreadData;
16
17 void* apply_sepia(void* arg) {
18     ThreadData *td = (ThreadData*)arg;
19     for (int i = td->start_row; i < td->end_row; i++) {
20         for (int j = 0; j < td->width; j++) {
21             int pixel_index = (i * td->width + j) * td->channels;
22             unsigned char r = td->data[pixel_index];
23             unsigned char g = td->data[pixel_index + 1];
24             unsigned char b = td->data[pixel_index + 2];
25
26             int nr = (int)(0.393 * r + 0.769 * g + 0.189 * b);
27             int ng = (int)(0.349 * r + 0.686 * g + 0.168 * b);
28             int nb = (int)(0.272 * r + 0.534 * g + 0.131 * b);
29
30             td->data[pixel_index] = (nr > 255) ? 255 : nr;
31             td->data[pixel_index + 1] = (ng > 255) ? 255 : ng;
32             td->data[pixel_index + 2] = (nb > 255) ? 255 : nb;
33         }
34     }
35     return NULL;
36 }
37
38 int main() {
39     // ...
40 }
```

```
acer@Ashmit:~/Workshop10$ gcc Q4.c -o Q4 -lpthread -lm
acer@Ashmit:~/Workshop10$ ./Q4
Process complete. Sepia filter applied to input.jpg -> output_sepia.jpg
acer@Ashmit:~/Workshop10$ gedit Q4.c
```

Image Before:



Image After:

